

Contents

Finding Memeland

Abstract

1. The Opportunity

2. The Game

3. The AI Stack

4. Tokenomics

5. Roadmap

6. Risks and Disclaimers

Contact

1

1

2

2

7

9

10

11

12

Finding Memeland

The First Autonomous AI Treasure-Hunt Memecoin on Base

Litepaper v0.9 — September 22, 2026

Abstract

Finding Memeland is a memecoin powered by an autonomous AI agent that runs a recurring, on-chain treasure hunt. On each hunt, the agent goes looking through the chains and chooses a **1/1 NFT that already exists** — made by somebody else, owned by somebody else, minted long before the project existed. It is never minted by us, never bought, never held, never touched. **The prize is always on Base. The treasure can be anywhere onchain.** The agent drops progressively more revealing clues on the project’s public channel, and waits. The first hunter who decodes the treasure’s name, finds the exact token, and posts it publicly — as a reply to the hunt’s opening post, either as a marketplace link or as `chain:contract:tokenId` — wins. Anyone can win; **holding \$FIND sets the size of the win**: a wallet that has held the public floor through the holding window takes 100% of the announced pot, while a non-holder winner takes 10% of it. The agent answers the winning claim right there on the thread, asks for the winner’s Base wallet (same account, ten minutes on the clock), and pays the prize automatically on-chain in \$FIND, the project’s native token. The treasure itself never moves: it stays with its owner, who was never part of the transaction.

The game itself runs with no human in the loop: target selection, clue generation and pacing, publishing, winner validation and prize disbursement are all executed by the agent. Each hunt opens with a published **commitment** that cryptographically binds the agent to one specific token — `SHA-256(target_id + metadata_sha256 + salt)` — before the first clue drops, and anyone can verify it once the hunt ends. The deeper guarantee is structural rather than cryptographic: **because the treasure belongs to a stranger, the project has no lever to pull.** It cannot edit the token, move it, or swap it for an easier one. The human operator provisions infrastructure and decides *when* hunts fire; the operator and associates are ineligible to win, never see a clue before it publishes, and cannot move the goalposts mid-game.

AI agents built by users to *help solve* clues are explicitly welcome; accounts that publicly identify themselves as bots or agents are not eligible to win. This split —

agents help, humans win — keeps the game accessible to solo players while turning each hunt into a public competition over tooling.

Finding Memeland launches on Base via a **Doppler multicurve auction**: a pure fair launch with an ascending price curve, permanently locked liquidity and zero team allocation. Trading fees fund ongoing operations and prize-pool top-ups in perpetuity; a 20% treasury allocation, vesting linearly over 180 days, funds the hunts.

1. The Opportunity

The two largest narratives in crypto consumer culture today — autonomous AI agents and memecoins — have until now grown in parallel rather than together. Agent platforms (Virtuals, Clanker, Bankr) have produced billions of dollars in trading volume and tens of millions in protocol fees, while memecoins continue to be the dominant entry point for new retail capital. Yet the two have mostly remained separate categories: agents are infrastructure stories, memecoins are pure speculation.

The gap is product. Most memecoins offer holders nothing to do after they buy. Most agent tokens offer holders impressive technology but little reason to interact with the agent personally. Holders churn out within weeks because the token does not produce ongoing surface area for attention.

Finding Memeland sits in that gap. The token is the access pass to a recurring AI-run game; the agent is the engine that creates recurring reasons to hold, engage and share. **The product is the loop, not the chart.**

The timing is favorable. Clanker has processed over \$50M in cumulative protocol fees since late 2024 and an \$8M+ week in February 2026 alone. Virtuals reports \$75M+ cumulative revenue with 18,000+ agents deployed. Base remains the lowest-friction L2 for consumer crypto, with Coinbase distribution and Farcaster as a native social layer. The infrastructure for autonomous agent-driven economies is, for the first time, production-grade and at scale.

2. The Game

Core loop

Each hunt is a complete, self-contained game. The treasure is not ours, and never was.

1. **The target.** The agent chooses a **1/1 NFT that already exists somewhere onchain** — made by someone else, owned by someone else, minted long before we arrived. We never mint it, never buy it, never hold it, and never touch it. The hunt happens *around* the piece while it sits exactly where it has always been. Candidates are found off the clock by a continuous search across the chains and held, encrypted, in a store the operator cannot read; each candidate is re-verified against the chain before it is ever used. **The prize is always on Base. The treasure can be anywhere onchain.**

2. **Sealing, the day before.** One candidate is drawn and sealed ahead of the hunt: the agent reads its metadata and artwork, writes Clue 1, has the clue checked, and computes the hunt's commitment. All of it happens before anybody knows a hunt is coming. This matters for a reason players feel rather than read: when the hunt opens, Clue 1 publishes in seconds instead of keeping an audience waiting on a machine that is still thinking.
3. **Clue 1 — the opening post.** The agent publishes Clue 1 on the main @FindingMemeland account. This post does four things at once: it announces the hunt is live, it carries the first oblique puzzle thread, it is the **reshare gate** — to be eligible to win this hunt, your X account must have reshared this exact post (retweet or quote-tweet) — it is the **claim window**: every claim in the hunt is a public reply to this exact post — and it carries the hunt's **commitment** (see *Provable integrity* below). Each hunt has its own reshare gate; a reshare of a previous hunt does not carry over.
4. **Continued clues.** The agent drops further clues on the main account at random intervals — tens of minutes to hours, unpredictable by design. The game is built on a **puzzle doctrine**: the opening clues are hard *pieces*, not steps — each one a single oblique constraint on the treasure's name or artwork that cannot yield the answer alone, but that a solver can check against a candidate; the pieces intersect on exactly one token. Before publication every clue passes three independent checks: a **blind AI solver** that sees only the clue and must fail to guess the answer; a **consistency judge** that reads the clue next to the answer and rejects anything false or pointing away from it; and a **search guard** that types the clue into a marketplace search and rejects it if the target surfaces — a clue must be solvable by thinking, never by pasting. After the puzzle phase the clues ease progressively until someone cracks it. If a hunt remains unsolved for **72 hours**, it is publicly voided: the agent announces the end, publishes every ingredient of the commitment anyway, and the prize returns to the treasury. **A void is as verifiable as a win.**
5. **The hunt.** Hunters work the clues — solo, in groups, or with their own AI agents — to decode the treasure's name, then search it on an NFT marketplace to find the one token that matches every clue.
6. **The claim — public, on the thread.** When a candidate has found the token, they post it as a **public reply to the Clue 1 post**, in either of two forms: the **marketplace link** (any marketplace — OpenSea, Rarible, SuperRare, anything we can resolve) or the identity written out as **chain:contract:tokenId**. That is the entire submission: no DMs, no forms, no code to copy. Ordering is decided by the reply's own timestamp (with the post id as a millisecond tiebreak) — the race is **publicly auditable by anyone**, and copying an answer out of someone else's reply can never beat them: the earlier timestamp always wins. The reshare gate is enforced here and it is eliminatory. Each account gets **five guesses**. A reply we could not parse — a wrong format, a collection link instead of a token link, a URL our resolver could not read — **is not a guess**: the agent says so, returns the format, and the five stay intact. The principle is fixed and it is not negotiable: *we never tell a player they are wrong about something we could not read.*
7. **The wallet ask — 10 minutes on the clock.** When the first valid claim is settled, the agent replies publicly to the winning post: congratulations, and a request for the winner's **Base wallet (0x...)**. From the moment of that reply

the winner has **10 minutes** to answer — as a reply to the agent’s post, **from the same X account that posted the winning claim**. A wallet posted by any other account is ignored outright: the binding is enforced on the account’s immutable numeric id, which is what makes a public wallet hand-off theft-proof. A malformed or checksum-invalid address gets one correction request (the clock keeps running). If the window expires, the agent publicly posts that the claim timed out and it lapses: the earliest valid claim waiting in line gets the ask next. **Miss your window — next in line.**

8. **Validation and payout.** Before any prize moves, the agent validates the full set: (a) the claimed token is the committed target; (b) the winner’s account re-shared Clue 1; (c) the account is not publicly identified as a bot or AI agent, and bot-signal screening passes. Holding is checked last — and it is **not eliminatory**: the agent proves directly from the chain (see *Eligibility* below) whether the winner’s wallet held the public floor continuously through the holding window, and that proof sets the **size** of the payout — **100% of the announced pot for holders, 10% for non-holders** (and when the reduced share applies, the Winner Announcement says so, in the open). The first claim satisfying the checks triggers an on-chain transfer of the prize in \$FIND to the winner’s address — and ends the hunt at that instant.
9. **Reveal.** Once a winner is paid, the agent publishes a Winner Announcement naming the treasure, crediting its artist where the metadata names one, linking to its marketplace page, showing the prize transaction hash — and the ingredients of the commitment, so anyone can recompute it. And it states the thing that makes this game different from every other treasure hunt: **the treasure stays with its owner**. It was never transferred, never bought, never at risk. The prize was on Base; the treasure was wherever it was.

Provable integrity

Every hunt opens with a cryptographic commitment. At sealing time the agent — autonomously — selects the target, hashes its metadata, generates a secret salt, and computes:

```
commitment = SHA-256(target_id + metadata_sha256 + salt)
```

```
where target_id      = chain:contract:tokenId    (e.g. ethereum:0x...:11385)
    metadata_sha256 = SHA-256 of the token's metadata JSON
                      (keys sorted, no ASCII escaping, utf-8)
```

The commitment is published inside the Clue 1 post, before any other clue exists. When the hunt ends, the Winner Announcement reveals the target id, the metadata hash and the salt. Anyone can recompute the value and confirm it matches Clue 1. This proves the target — one specific token, on one specific contract, on one specific chain — was fixed before the first word of the first clue, and that its metadata was what we say it was.

And there is a second lock this game has that the old one did not: the treasure is not ours. It belongs to a stranger who has never heard of us. We cannot edit it, cannot move it, cannot quietly swap it for an easier one, and cannot arrange for a friend to own it. The strongest guarantee in the protocol is not a hash — it is the absence of a lever.

That independence cuts both ways, and we are explicit about it. A third party can

change or burn the token mid-hunt, because it is theirs to change or burn. So the agent **reads the target from the chain before every single clue**. If the token's metadata has moved or the token is gone, the hunt does not quietly continue: it is voided and the readings are published, the same ingredients as a win. And if the change happens *after* a valid claim, the winner is paid in full and the change is noted publicly — the commitment binds the token's identity and its metadata at launch, never who owns it or what a third party does to it later.

Three further commitments back this up. **Insider ineligibility:** the operator and anyone associated with the project cannot win, ever. **Operational blindness:** the system is designed so the operator does not see the target's identity or any clue before it publishes — candidates are stored encrypted, sealed blind, and every game post publishes autonomously with no human approval step. The operator's own status tools report counts, hours and verdicts, and are written to never print a name. **Auditable submission log:** the agent logs every submission it processes, with timestamps and per-filter outcomes. And because claims are public replies, the **ordering itself is publicly verifiable** — the winning timestamp sits on the thread, next to everyone else's, for anyone to check. No trust in the agent's clock required.

We are explicit about the limit of these guarantees: cryptography proves the target was fixed in advance; it cannot prove what no human ever saw. Closing that final gap by running the agent inside a trusted execution environment (TEE) is on the roadmap.

Eligibility, anti-sniper, anti-bot

Three checks apply to every submission — two eliminatory gates (the public reshare and the bot screen) and one on-chain holding proof that is not a gate at all: it sets the size of the prize.

Holding floor — hold to win it all. Anyone can win a hunt; **holding is what turns a win into the full bounty**. A winner whose wallet has held a minimum balance of \$FIND continuously through the holding window takes **100% of the announced pot; a non-holder winner takes 10%**. The split is stated in every Clue 1 post — nobody discovers the rule after winning. The floor is a **standing, public token amount — currently 10,000,000 \$FIND** — published and in force at all times, deliberately decoupled from hunt timing so hunts stay unpredictable. As the market cap grows, the project announces **reductions** to the floor, keeping the dollar cost of the full share accessible; the floor only ever moves down, so nobody who holds the floor today loses their full share tomorrow. Continuity is not sampled or estimated: it is **proven, exactly, from the chain**. When a claim arrives, the agent reads the wallet's current balance and replays every Transfer event that touched the wallet inside the holding window, reconstructing its balance at every point in time. If the balance dipped below the floor for even one block, the wallet counts as a non-holder for that hunt — the win stands, at the reduced share.

Holding window — 24 hours. To take the full pot, your wallet must have held the floor balance continuously for **at least 24 hours** before you claim. This single, flat rule is the project's core **anti-sniper** filter, and it exists for one reason: to keep the game honest. Without it the optimal play is to buy mid-hunt, win, and sell the same evening — a game that pays people for arriving late. With it, the optimal play is to buy before hunts and stay. The Transfer-event replay proves continuity to the block, so a last-minute buyer wins at the non-holder share, never the holder's; the same economics blunt automated farms, since a disposable wallet spun up mid-hunt only

ever plays for 10% of the pot. The window is sized to outlast any hunt — hunts void at 72 hours but are solved in hours — so buying after Clue 1 drops cannot produce a full-prize claim in the same hunt. Changes to this rule are announced publicly and in advance, never applied to a hunt already running.

Public reshare. The winner’s X account must have reshared (retweeted or quote-tweeted) Clue 1 of the current hunt — and the reshare must already exist when the claim is checked: claim first, repost later doesn’t count (the claim is rejected and must be re-posted, with a new timestamp). This converts every participant into a distribution node for that specific hunt, and it forces each candidate to expose a real, persistent X identity that has interacted publicly with the project.

Bot defences. The primary defences are structural: ordering is decided by public post timestamps (speed of automation cannot rewrite a timestamp), the proven-holding window caps what a disposable wallet can ever earn at the non-holder share, the per-account guess cap blunts brute force, and the reshare gate forces every candidate to expose a persistent public identity. On top of that, every submission passes the bright-line self-identification screen described below, and the operator holds a kill switch that can pause a hunt — publicly disclosed — if abuse is detected mid-game.

Humans win, agents help

Building AI agents to help you solve clues — image analysis, context scraping, candidate scoring, pattern-matching across posts — is not only allowed but encouraged. The agent-builder community is part of the project’s culture, and tooling competition makes the game better over time. **What the rule protects is the identity of the winner:** the account that claims the prize must not itself be an AI agent or bot.

The project enforces this through a clean, externally-verifiable test: **accounts that publicly identify as agents or bots cannot win.** Specifically, the following are ineligible regardless of speed or holding: (a) accounts with terms like “bot”, “agent”, “AI” or equivalent in their display name, handle or bio — screened automatically on every submission; (b) accounts that publicly present as automated elsewhere (X’s “Automated” label, registration as an autonomous agent on platforms such as Virtuals or Bankr) — enforced as policy, by review; and (c) Finding Memeland’s own main and operational accounts. This rule is objective and does not require us to judge whether a given account is “really” human.

Beyond that bright-line rule, ambiguous cases fall to review: account age and organic posting history, public reshare interaction with Clue 1, and submission patterns. The operator can pause a hunt before payout where signals conflict — pauses are disclosed publicly — and the project reserves the right to disqualify a submission and award the prize to the runner-up where there is convincing evidence of fully automated submission.

The intent is plain. If you are an attentive solo human, you can win. If you are a builder who points your own private agent at the clues and then submits from your own personal account, you can win — and likely faster. If your account is publicly an agent, or you are running a covert farm of freshly-spun accounts, you will not.

Why this works

A memecoin succeeds when it produces recurring attention loops. Finding Meme-land engineers them at three levels: each clue drop is shareable content, the search is collaborative on social media, and the agent-builder community treats every hunt as a public competition over tooling. Anyone can play — holding the token is what multiplies the win by ten. The per-hunt reshare gate ensures each hunt creates its own organic distribution wave, without anyone having to ask for it: the prize asks on the project's behalf. The random cadence — hunts announced when they happen, clues at unpredictable intervals — keeps the game from settling into a schedule people can ignore, and the 72-hour void rule guarantees every hunt has an ending worth talking about.

And the premise itself is the marketing. Every hunt ends with an artist being credited and a piece of existing onchain art put in front of an audience that had never seen it — at no cost to its owner, who was never part of the transaction. The game does not manufacture its treasure. It goes looking for one that was already there, and pays somebody for finding it.

3. The AI Stack

The agent is built as a modular pipeline. The game loop — identity generation, clue generation, publishing, validation, payout — runs autonomously, with no human approval on any game action. The human operator provisions infrastructure (accounts, funds), triggers hunts, and can halt the system; the operator cannot win and does not see game content before it publishes.

Target finder & larder. A continuous search walks curated sources across several chains and tests each candidate token against the chain itself: metadata readable and content-addressed, a name distinctive enough to be findable by honest search but not already famous, artwork our pipeline can actually read, and an owner that is a plain wallet rather than a contract. Survivors are kept **encrypted** in a store the operator cannot read — the *larder* — and every one of them is re-verified against the chain before it is ever used. Searching off the clock is what lets a hunt open in seconds instead of making an audience wait.

Search guard. Before any hunt fires — and again at launch — the agent runs the attack it defends against: it types the clue into a public marketplace search and checks whether the target surfaces. A canary query runs first (searching the target's own on-chain name, which *must* return it) so a silent index outage can never be mistaken for a clean result. A clue that can be pasted into a search box is not a clue, and it does not publish.

Clue engine. Plans a clue trajectory that resolves to the target and distributes drops at random intervals. The opening phase is a fixed number of hard *puzzle pieces* — each attacking the name or the artwork from a distinct assigned angle, one constraint per clue — followed by a reveal phase that eases until the hunt is won (or the 72-hour void deadline passes). Four layers of QA run before anything publishes: deterministic guardrails verify that no clue literally contains the answer (including emoji, which are read by name); a **structural checker** verifies every claim a clue makes about the name's letters, so a clue can never state something false that would send a correct solver away from the answer; a **blind AI solver** — a different model from the writer, shown only the clue — must fail to guess it; and a **consistency judge** reads the clue next to the answer and rejects anything false or

pointing elsewhere. A clue that cannot clear QA is regenerated, never published.

Content publisher. Posts to X through the official API. Farcaster publishing is planned as the community grows.

Claim reader & validator. Reads the public claim thread continuously through the X mentions timeline (a reply to Clue 1 mentions the main account by construction), with full pagination and a periodic full-thread sweep so replies-to-replies are seen too. Shortened links are expanded back to their real URLs before anything is parsed. Claims are ordered by their public timestamps, never by processing order: before any winner is called, a settlement window re-reads the whole thread, so a claim the platform delivered late still wins if its timestamp is earlier. A claim may arrive as a **marketplace link from any site** or as `chain:contract:tokenId`; links that carry a token but no chain are resolved against the chains themselves rather than guessed at. For each claim it runs the eligibility checks (target match, reshare-at-processing, bot screening including the public-self-identification rule), publicly asks the winner for their Base wallet, accepts it **only from the same account id** that posted the winning claim, checksum-validates it on the spot, and proves holder status on-chain via Transfer-event replay (Base RPC) — the proof that decides whether the payout is the full pot or the non-holder share. Anything it could not read is answered with the format and **costs the player nothing**: the agent never tells someone they are wrong about something it failed to parse. The 10-minute wallet window, the pending winner and the claim queue are persisted — a restart mid-claim resumes exactly where it stopped. Every attempt is logged with timestamps and per-filter outcomes; wrong guesses get rate-capped public jeers generated on the fly, hard-filtered so a reply can never leak a clue.

Live target check. The treasure belongs to somebody else, so it can change while the hunt runs. Before every clue the agent reads the token from the chain — through rotating public providers, so no single provider ever learns which token is being watched — and compares it to what was sealed. Intact: the hunt continues. Changed or burned: the hunt is voided and the readings are published. Changed *after* a valid claim: the winner is paid in full and the change is noted publicly. And when the read itself fails, the agent holds rather than concludes — an outage on our side is never reported as a fact about the target.

Payout executor. Sends the prize from a dedicated hot wallet that holds only enough for one to two hunts at any time, with a hardcoded per-hunt cap. Payouts are **idempotent by construction**: the agent journals a payout intent before broadcasting any transaction, and at most one transfer can ever execute per hunt — a crash, restart or retry can never pay twice. The bulk of the treasury sits in a separate Safe on Base controlled by the operator; the agent has no access to it. A worst-case compromise of the agent is bounded by the hot wallet's balance.

Reliability layer. Once a hunt is live, people are playing — so the game loop is engineered never to die mid-hunt. Every phase (DM reading, validation, clue publishing) is isolated: transient failures are retried with backoff and alerted to the operator, a submission that cannot be processed is retried without losing its place in the arrival order, and a failed clue skips one round instead of ending the game. If the process itself is restarted, the agent **resumes the live hunt exactly where it stopped** — same target, same commitment, same reshare gate, same submission marker — from state persisted on every step. Before any hunt fires, a pre-flight check verifies the AI services, the X API, the RPC, and the money itself: gas in the hot wallet and a token balance sufficient for the prize. The operator additionally

holds a kill switch that pauses the game (no clues, no processing, no payouts — public claims keep accumulating on the thread with their timestamps preserved, so pausing never changes who was first); pauses are disclosed publicly.

Hunts are triggered manually by the project admin via a private command channel rather than running on a fixed schedule. This is deliberate: it lets each hunt be coordinated with marketing pushes, partner announcements, or simply the rhythm of the community, instead of firing on autopilot.

4. Tokenomics

Parameter	Value
Token	\$FIND — Finding Memeland
Network	Base
Launch platform	Doppler multicurve auction (direct deployment)
Total supply	100,000,000,000
Liquidity	80,000,000,000 (80%) — sold on the multicurve; pool liquidity permanently locked
Prize Treasury allocation	20,000,000,000 (20%) — to the Treasury Safe, linear vesting over 180 days
Team allocation	0%
Trading fee	1% per swap
Fee split	95% Project Treasury / 5% Doppler Protocol
Treasury custody	Safe (multisig) on Base — signer set and threshold published at launch

Launch model

Finding Memeland launches via a **Doppler multicurve auction**, deployed directly by the project on Base. The multicurve model sells the 80% liquidity allocation along an ascending sequence of bonding curves: the price can only step upwards as demand fills each curve, there is no presale, no allowlist and no team bag, and the resulting pool liquidity is **permanently locked**. The opening valuation is set deliberately low, so the earliest believers get the fairest entry the mechanism allows.

The remaining 20% — the Prize Treasury allocation — vests **linearly over 180 days** to the project's public Safe. Nothing unlocks in a cliff; roughly 0.55% of supply becomes available per day. This is a deliberate anti-rug structure: the treasury physically cannot dump its allocation, and the vesting stream doubles as a runway discipline for prize funding. Hunt #1 fires within the first week after launch, on a date announced in the Genesis countdown — and with the flat holding rule (see §2), anyone can win it — holding the floor through the 8-hour window is what takes the full pot.

Fee economics

Every swap of \$FIND on the launch pool generates a 1% fee. Of that fee, **95% is routed to the Project Treasury** at a publicly-disclosed Safe address; the remaining 5% flows to the Doppler protocol. The 95% share is the project's recurring operational revenue and accrues continuously in proportion to traded volume — without any sale of treasury tokens. As long as the game generates trading volume (which is the whole point of the design), the project remains self-funding.

Treasury policy

The Treasury is held in a Safe on Base. The signer set and threshold are published at launch and upgrades to the signer policy are announced in advance.

Trading fees accrue in both currencies of the pool — ETH and *FIND*, *in proportion to swap direct*
ETH—side fees fund the project : *AI and infrastructure operating costs and pre-*
announced development. * *FIND-side fees flow to the prize treasury**: they fund hunts, in full. Every ETH fee runs the machine; every \$FIND fee goes back to the players. The vesting 20B allocation seeds the prize treasury from launch.

During the growth phase, prizes are deliberately treated as growth spend: the first hunts are funded to grow the game and its community, not to maximize treasury retention. The Treasury will not be used for founder compensation outside pre-announced policy, undisclosed token sales, or unrelated investments.

Prizes are denominated **directly in \$FIND** — a fixed token pot set when each hunt fires and announced in the Clue 1 post, with a standing public minimum per hunt. Holders win the full pot; a non-holder winner receives 10% of it, and the remainder stays in the prize treasury for future hunts.

All Treasury movements are visible on-chain. The Safe transaction history is the project's public ledger.

Why no team allocation

A pre-allocated team bag would undermine the project's primary credibility claim — that the agent and the community own the game, and that the founder benefits only through the same fee mechanism that benefits the protocol and the holders. We accept the trade-off: the founder has no token upside outside of treasury-funded compensation that is itself bounded by public policy.

5. Roadmap

Phase 0 — Build (May - July 2026). AI stack assembly and red-team testing across all modules — including a full independent code audit (game mechanics, security, money paths) and live end-to-end rehearsals of the complete hunt on X. Hunt Integrity Protocol implementation (commitment hash, submission log, operational blindness). Crash-resume, payout idempotency and kill-switch hardening. Brand, voice and primary channel (X; site). Legal review of the game mechanics and token issuance. Pre-launch community seeded through the Farcaster and Base agent-builder networks.

Phase 1 — Launch & First Hunts (August 2026). Deployment via Doppler multicurve on Base. Public verification of vesting parameters, treasury address and signer set. Genesis countdown, Hunt #1 within the first week. Prize size, eligibility rules and clue cadence published from day 0.

Phase 2 — Cadence and Growth (Q4 2026 - Q1 2027). Regular cadence of hunts, triggered to align with marketing pushes. Public leaderboard of winners. Public submission-log dashboard. Open APIs for community-built solver agents. Partnerships with Farcaster channels and Base ecosystem voices. Monthly treasury transparency post. Exploration of TEE deployment for verifiable agent autonomy, and a unified public collection for the game’s trophies and companion pieces.

Phase 3 — Memeland-as-a-Service (Q2 2027+). Once the game has audience and a proven format, external projects, companies and product launches can commission custom hunts featuring their brand — the agent runs the hunt, their product is the target, our community is the audience. Clients pay a service fee; a portion of every fee is used to acquire \$FIND on the open market to top up the prize treasury, so platform revenue feeds directly back into the game economy. This converts Finding Memeland from a single-token game into a marketing-as-a-game platform. The fee split and mechanics will be published before the first commissioned hunt and are subject to the legal guidance described in §6.

The exact schedule will adapt to market conditions and execution. Roadmap revisions will be published transparently.

6. Risks and Disclaimers

Platform risk. The hunt’s *target* lives on-chain and was never ours to begin with — no platform can suspend, edit or delete it, and neither can we. The flip side is that its owner can: the agent therefore re-reads the token before every clue and voids the hunt publicly, with the readings, if it changed. What still depends on X is the megaphone: clues, claims and payouts run through the project’s X account and API. Account suspension or API policy changes could interrupt hunts; the project maintains contingency procedures (published hunt-abort rules, community channels outside X) and can re-run an interrupted hunt on a fresh target.

Agent failure risk. The autonomous agent can produce unintended outputs (off-brand clues, scheduling errors, validation edge cases). Pre-publication guardrails and the blind-solver QA, the phase-isolated reliability layer, crash-resume and the human admin’s kill switch bound the damage. Pauses are disclosed publicly.

Integrity limits. The integrity hash proves each hunt’s target was fixed before the first clue; it does not, by itself, prove that no insider ever saw it. The project mitigates this with insider ineligibility, operational blindness by design, auditable submission logs — and, on the roadmap, TEE deployment for externally verifiable autonomy. Users should weigh these guarantees for themselves.

Regulatory risk. \$FIND is positioned as a utility token providing access to a skill-based game. The project is obtaining legal guidance on the game mechanics and on token issuance requirements in its operating jurisdiction (including EU MiCA), and will restrict participation where legally required. The project does not promise investment returns and is not registered as a financial product. Users are responsible for understanding the rules of their own jurisdiction.

Market risk. Token prices are volatile. Trading volume can dry up, which would reduce the fee stream that funds operations. The vesting 20% allocation provides initial runway, but is not infinite.

Custody risk. The treasury Safe and the agent's hot wallet are described in §3 and §4; the agent can never access more than the hot wallet's balance. We treat custody as an ongoing security exercise, not a one-time setup.

This document is not financial advice and not an offer to sell securities. It describes a product and its mechanics. Read carefully, do your own research, and risk only what you can afford to lose.

Contact

- X: [@FindingMemeland](https://x.com/FindingMemeland)
- Telegram: t.me/findingmemeland
- Site: findingmemeland.com

Litepaper v0.8 — August 29, 2026. This is a living document. Material changes will be versioned and dated.

Changelog v0.8 → v0.9

- **The treasure is no longer ours — and never was.** The agent stopped inventing and minting a relic. It now chooses a **1/1 NFT that already exists** somewhere onchain, made and owned by strangers, and never mints, buys, holds or touches it. The prize is on Base; the treasure can be anywhere. This is the single largest change in the project's history and it replaces a cryptographic promise with a structural one: the project has no lever to pull on a token it does not own.
- **Commitment v2.** `SHA-256(target_id + metadata_sha256 + salt)`, published in Clue 1 and revealed at the end. It now binds the token's *metadata* as well as its identity, so "the answer never changed" covers what the piece actually was, not only which token it was.
- **The target is read from the chain before every clue.** A third party can change or burn a token that belongs to them. Intact, the hunt continues; changed or burned, the hunt is voided with the readings published; changed after a valid claim, the winner is paid in full and the change is noted. When our own read fails, the agent holds instead of concluding.
- **Claims accept marketplace links from any site.** A claim is the marketplace link or `chain:contract:tokenId`. There is no claim code to find any more, because there is no description we control. A reply the agent cannot parse costs the player nothing and is answered with the format — the agent never tells someone they are wrong about something it failed to read.
- **Five guesses per account, and the count is spoken aloud.** A real wrong guess spends one and the reply says how many remain.
- **Search guard.** Every clue is typed into a public marketplace search before it publishes, behind a canary query that proves the index is answering. A clue that can be pasted into a search box does not publish.

- **Structural checker and consistency judge.** Two new QA layers: every claim a clue makes about the name's letters is verified against the name, and a judge reads each clue next to the answer to reject anything false or pointing away from it.
- **Sealing moved to the day before.** The target, its description and Clue 1 are prepared and judged ahead of time, so a hunt opens in seconds rather than keeping an audience waiting.
- **Holding window: 8 hours → 24 hours.** The old window let a mid-hunt buyer take the full pot and sell the same evening. Announced publicly in advance; never applied to a hunt already running.
- **Relics retired.** The relic model — invented beings minted as 1/1s, transferred to winners as trophies — is discontinued. Hunts #1-#10 played under it and that history stands; the changelog entries below are kept as the record of it.

Changelog v0.7 → v0.8

- **The hidden target moved on-chain: personas became relics.** The agent no longer hides behind a disposable X account; it invents a fictional being and mints it as a 1/1 NFT on Base — name, AI artwork, lore, and the hunt's claim code written into the on-chain description. Hunters decode the two-word name from the clues and find the relic on a public NFT marketplace. Validated in production across two live hunts before this revision.
- **The target is now physically immutable.** The relic's contract ownership is renounced at mint, so nothing about it can be edited mid-game — by anyone. The integrity hash now commits to `chain:contract:tokenId`, binding one specific token; "the treasure never moved" became a property of the chain rather than a promise.
- **The winner keeps the relic.** Alongside the \$FIND pot, the relic NFT itself is transferred on-chain to the winner — a one-of-one trophy with the hunt in its provenance, freely tradable. Past relics form a growing on-chain gallery of the game's history.
- **Why the change.** It removes the game's biggest platform dependency (single-use X accounts: provisioning, warm-up, indexing delays, suspension risk), strengthens the integrity guarantees (immutable target, verifiable to the token), adds a second prize dimension with real secondary-market value, and lets the game scale to a much higher hunt cadence.
- **Clue engine hardened.** The opening clues follow a strict puzzle doctrine — one oblique constraint per clue, distinct angles, no rhymes, no emoji, no giveaways — and every puzzle clue must defeat a blind AI solver before it publishes. Difficulty is now enforced by proof, not by prompt.
- **Blind mint pipeline.** Relic identities are generated and stored encrypted; minting, findability checks and launches are operated through interfaces that never display the name, artwork or code. Fresh, unlinkable wallets mint exactly one relic each.

Changelog v0.6 → v0.7

- **Holding is no longer eliminatory — it sets the size of the prize.** Anyone can win a hunt. A winner whose wallet held the public floor continuously through the holding window takes **100% of the announced pot; a non-holder winner takes 10%.** The split is stated in every Clue 1 post, and the Winner Announcement discloses when the reduced share applies. Validation

order changed accordingly: holding is proven last, as a payout modifier, after the eliminatory gates (claim code, reshare, bot screen).

- **Holding window shortened: 24 hours → 8 hours.** Sized to outlast a typical hunt (so a mid-hunt buy still cannot earn the full pot in the same hunt) while making the full share accessible to anyone who genuinely holds. The floor is published: **10,000,000 \$FIND** — and, as before, it only ever moves down.
- **Prizes denominated directly in \$FIND.** Each hunt's pot is a fixed token amount announced in Clue 1 (replacing the USD-denominated range converted at fire time). A standing public minimum pot applies.
- **Clue trajectory re-specified from production data.** The early clues now cycle the persona's profile facets — name and face — in two passes, hard first, clearer second, never the same facet twice in a row; clues pointing at the persona's own posts only begin at clue 7. Cadence wording updated to match production pacing (random intervals, minutes to hours, tuned per hunt).

Changelog v0.5 → v0.6

- **Claims moved from Direct Messages to public replies on Clue 1.** In production testing, X's DM API proved structurally unreliable for reading players' messages (a platform-side limitation, not fixable on our end). The redesign is an upgrade, not a workaround: submissions are now public replies on the Clue 1 thread, ordered by their own public timestamps — so the race is auditable by anyone, sniping a code from another player's reply can never beat the original, and the entire hunt (claims, the oracle's jeers, the win, the payment proof) lives in one public thread.
- **The wallet hand-off is public and timed.** The agent answers the winning claim on the thread and asks for the winner's Base wallet: **10 minutes from the agent's reply**, answered from the **same account** that posted the winning code (enforced on the account's immutable numeric id — a wallet posted by anyone else is ignored). Checksum-invalid addresses get one correction ask. Miss the window and the claim lapses publicly: next valid claim in line gets the ask; if the line is empty, the hunt reopens and the next valid code wins.
- **Reshare gate timing made explicit:** the reshare must exist when the claim is checked; code-then-repost requires posting the code again (new timestamp).
- **Anti-spam hardened:** one oracle jeer per account, five code-like guesses per account per hunt, global reply budget — and the reply engine is architecturally unable to leak clues (it never receives clue content).
- **End-to-end validated in production** before this revision: a full live rehearsal ran the complete flow — wrong code, missing repost, theft attempt on the wallet ask, timed wallet hand-off, on-chain payout and integrity reveal — with every rule enforced as written here.

Changelog v0.4 → v0.5

- **Launch platform: Doppler multicurve (direct deployment)** replaces "Clanker v4 via Bankr". Bankr migrated its launcher away from Clanker, and the Doppler multicurve model won on the merits: ascending-price fair launch, permanently locked liquidity, and a 95/5 fee split in the project's favour (vs 40/60). Supply and split unchanged: 100B, 80% liquidity / 20% prize treasury / 0% team.
- **Prize treasury vesting: linear over 180 days** replaces the 7-day lockup + instant vest. Anti-rug by construction (~0.55% of supply per day) and a built-in

runway discipline. Hunt #1 timing is now set by the Genesis countdown (first week), no longer tied to a vault unlock date.

- **Anti-sniper check upgraded from sampling to exact on-chain proof.** Holding continuity is now proven by replaying the wallet's Transfer events across the full window — exact to the block, no sampling gaps. A dip below the floor at any point disqualifies; the litepaper's promise got stronger, not weaker.
- **72-hour void rule added.** A hunt unsolved after 72 hours is publicly voided; the persona keeps its secret and the prize returns to the treasury.
- **Retired personas keep their dressed profiles** as public artifacts of each hunt — a growing, verifiable gallery of the game's history. (Accounts still never play twice.)
- **Reliability layer documented:** phase-isolated game loop that survives transient failures, crash-resume of live hunts from persisted state, idempotent payouts (at most one transfer per hunt, journaled before broadcast), money pre-flight (gas + prize balance checked before launch), and a public-disclosure kill switch.
- **Bot-defence and submission-log language aligned with the shipped system:** the bright-line self-identification screen is automated; label/platform-registration rules are enforced as policy by review; the hunt ends at the first valid claim (later submissions are not processed); the full submission log is retained and auditable, with the count published at hunt close and a public dashboard in Phase 2.
- **Wallet checksum validation at submission time** — a mistyped address is caught when submitted, not after winning.
- **Holding floor redefined as a standing public token amount**, decoupled from hunt timing (hunts stay unpredictable; you can buy anytime and be eligible 24h later). It only ever moves **down**: as market cap grows, reductions are announced publicly, keeping entry accessible while the cost of disposable wallets tracks the value of the prizes.
- **Ticker finalized: \$FIND.**